

PRÁTICAS DE EXTENSÃO DOS CURSOS DE COMPUTAÇÃO

Professora: João Vitor da Costa Andrade

Atividade (parte 2) desenvolvida na aula de Projeto Interdisciplinar III: PI V

Unidade curricular envolvida:

(EXTE230011) Práticas de Extensão I: Fundamentação e Justificativa



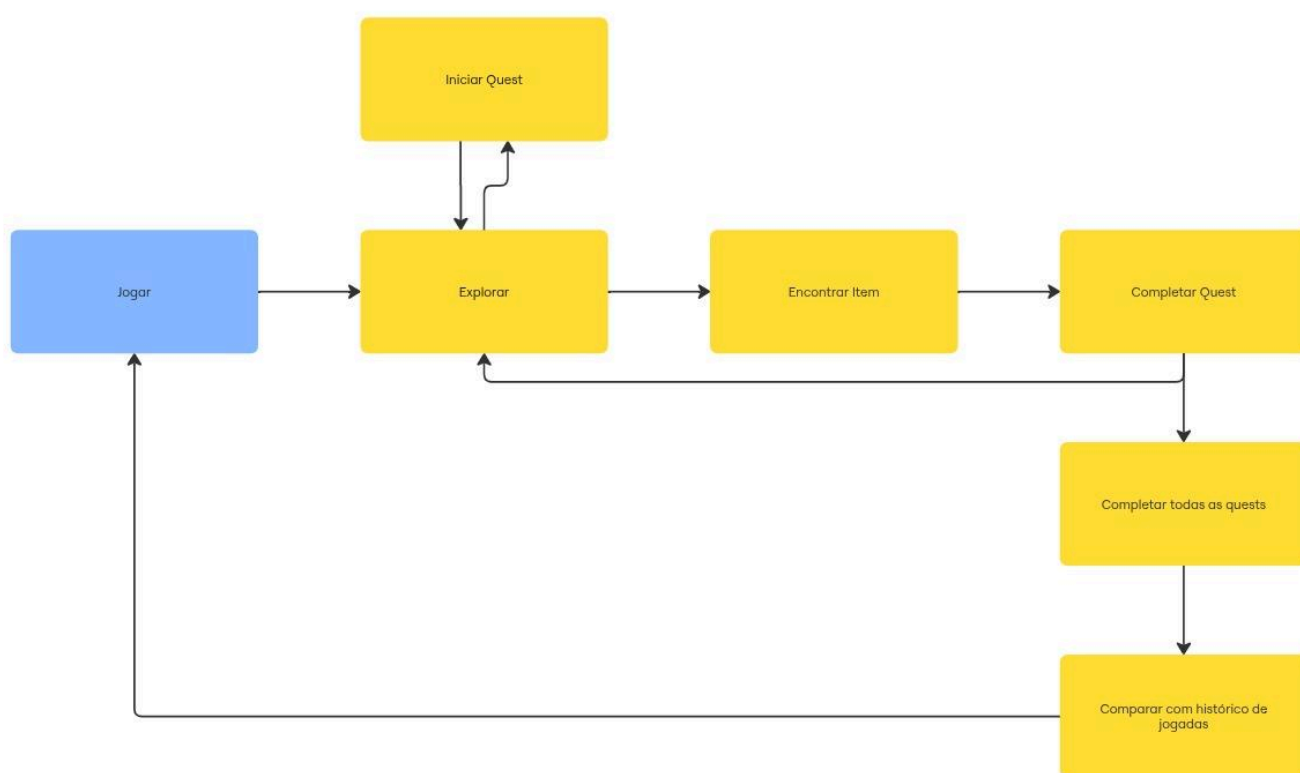
EcoRunner	
Discente	Matrícula
Marcos Yudi Kimura	251014039
Witor Tenã	251041043
Thiago Moreira Antoniassi	251041011
Rafael Eiji Hokama Utiyama	251014021

Design de Mecânicas e Lógica do Jogo: O objetivo desta etapa é transformar a ideia inicial do projeto em uma estrutura jogável consistente, com foco na definição das mecânicas e na organização lógica do funcionamento do jogo. A partir do que foi criado e projetado anteriormente (etapa 1) o grupo deverá simular o funcionamento do jogo manualmente, movimentando os elementos desenhados, de modo a verificar se a proposta é compreensível e funcional.

Segue link de vídeo da demo gameplay:

https://drive.google.com/file/d/1ARjSXQIL5B_pAYcVtWBI3R3G_BJViI0y/view?usp=sharing

Representar o ciclo do jogo: Evidenciar como se dá a dinâmica de repetição da experiência do jogador (sequência de passos/ esquema/ instruções). Esse ciclo deve contemplar o desafio apresentado ao jogador, a ação realizada por ele, o retorno ou feedback obtido e o reinício da interação. É importante que identifiquem, dentro desse fluxo, os momentos em que o jogador pode vencer, perder ou reiniciar a partida, garantindo que o jogo seja cíclico e rejogável.



Modelagem lógica do jogo: Explicitar os diferentes estados possíveis, como início, execução da partida, término por falha e conclusão por sucesso. Identificar as principais decisões envolvidas, como situações de erro ou de conquista de objetivos, bem como refletir sobre o que ocorre continuamente durante o jogo, caracterizando o seu funcionamento repetitivo. Também deverão considerar as ações do jogador, como comandos de teclado ou interações, relacionando essas entradas com as respostas do sistema.

1. Os Diferentes Estados do Jogo (Máquina de Estados Atualizada)

- **Estado 1: Início (Spawn e Tutorial)**
 - **O que faz:** O sistema inicializa o cronômetro da partida, carrega o mapa e "spawna" o personagem Leco no bioma do Planalto.
 - **Decisão Lógica inicial:** O jogo força o engajamento do jogador com o primeiro NPC para apresentar o ciclo básico (pegar o lixo no topo da montanha), servindo como um tutorial invisível.
 - **Estado 2: Execução (Exploração e Otimização)**
 - **O que faz:** O jogador transita por diversas áreas do mapa. O sistema gerencia simultaneamente a posição do jogador, a inteligência artificial dos inimigos no caminho, a detecção de obstáculos e a atualização contínua do tempo de jogo.
 - **Estado 3: Término por Falha (Permadeath)**
 - **O que faz:** Se as vidas do jogador chegarem a zero devido a colisões com obstáculos ou inimigos, o sistema altera para a tela de Game Over.
 - **Decisão Lógica:** Como o progresso é perdido, o sistema limpa completamente as variáveis de estado (zera o inventário, reseta o caderno de missões e o cronômetro) e obriga o jogador a recomeçar do zero.
 - **Estado 4: Conclusão por Sucesso (Speedrun e Placar)**
 - **O que faz:** Quando a última missão pendente for validada, o estado de execução é interrompido.
 - **Decisão Lógica:** O sistema congela o cronômetro, exibe a tela de vitória, salva o tempo final em um banco de dados local ou arquivo de save e o adiciona ao Placar (*Leaderboard*).
-

2. O Funcionamento Contínuo (O Ciclo de Jogo)

O *Game Loop* agora ganha uma camada extra de responsabilidade atrelada à nova dinâmica de missões. Durante a etapa de **Atualização Lógica (Update)** que ocorre dezenas de vezes por segundo, o sistema realiza as seguintes checagens contínuas:

1. **Monitoramento de Tempo (Timer):** O sistema incrementa continuamente o tempo de jogo.
2. **Ciclo de Missões:** O jogo entra no laço que você descreveu:

Procurar NPC -> Receber Quest -> Desviar de Ameaças -> Coletar Item -> Levar ao Local Específico.

3. **Monitoramento de Dano:** O sistema calcula continuamente as *hitboxes*. Qualquer erro de rota ou colisão não intencional com obstáculos subtrai recursos do jogador e pode levar ao reinício forçado.

3. Ações do Jogador e Respostas do Sistema (Feedback de UI)

Com a introdução do "Caderno de Missões", a interface de usuário (UI) passa a ter um papel fundamental na comunicação de dados ao jogador.

- **Completar uma Missão (Ação em Ambiente Específico):**
 - *Ação:* Jogador usa o item correspondente no ambiente exigido (ex: lixo na lixeira).
 - *Resposta Lógica:* O sistema altera o status daquela missão específica para **concluída** dentro da estrutura de dados que gerencia as missões (um *Array* ou *Lista Encadeada*, por exemplo).
 - *Resposta Visual 1:* O sistema renderiza um pop-up/aviso temporário na tela: "Quest Concluída".
 - *Resposta Visual 2:* O "Caderno de Missões" é atualizado internamente.
- **Consulta de Progresso:**
 - *Ação:* Jogador pressiona a tecla de abrir o Caderno de Missões (ex: tecla 'TAB' ou 'J').
 - *Resposta do Sistema:* Pausa a movimentação do jogo e renderiza a UI do caderno, filtrando e exibindo as listas de missões: **Não Começadas, Em Andamento e Concluídas**.

4. Principais Decisões (Árvore Lógica Final)

As regras de negócio do jogo formam a árvore de decisões que dita se o jogador falhou ou triunfou:

- **Lógica de Falha:**
 - SE (colisão == inimigo OU colisão == obstáculo) ENTAO vidas = vidas - 1
 - SE (vidas <= 0) ENTAO disparar_estado(GAME_OVER) e zerar_progresso()
- **Lógica de Conclusão de Missão:**
 - SE (jogador_possui_item_correto E colisao_com_ambiente_alvo) ENTAO status_missao = CONCLUIDA
- **Lógica de Condição de Vitória (Endgame):**
 - O sistema verifica a todo momento o Caderno de Missões.
 - SE (quantidade_missoes_concluidas == total_de_missoes_do_jogo) ENTAO disparar_estado(VITORIA) E registrar_tempo(Placar)